Database Design midterm study guide

CSCI 340 | Topic checklist and quick reference

The topics below are fair game for the midterm. Be able to explain the ideas, interpret small examples, and apply them to a new schema or set of requirements. The reference pages summarize important definitions, notation, and PostgreSQL syntax.

Exam format: 50 minutes, 60 points, with multiple choice, short answers, and longer query and modeling questions. You may use one standard letter-size sheet of notes, front and back, for syntax or definitions.

Relational model and keys

☐ Distinguish a relation, tuple, attribute, domain, schema, and instance; recognize that relations are unordered sets of tuples.

☐ Identify superkeys, candidate keys, primary keys, composite keys, and foreign keys from the stated rules.

☐ Explain referential integrity and why uniqueness in one sample does not establish a key for all valid instances.

Relational algebra

☐ Read and write selection, projection, rename, Cartesian product, natural join, and theta join expressions.

☐ Compose operators and trace the output schema and tuples, including duplicate removal by projection.

☐ Use union, intersection, and set difference on compatible schemas; explain the direction of difference.

Basic SQL queries

☐ Use SELECT, FROM, WHERE, aliases, comparisons, AND, OR, and NOT; group conditions with parentheses.

☐ Join two or three tables using JOIN … ON and identify which columns express the intended relationship.

☐ Use DISTINCT, ORDER BY, IS NULL, and IS NOT NULL; explain what one result row represents and why duplicates can appear.

Table definitions and constraints

DDL means Data Definition Language: SQL commands that define or change database structures, such as CREATE TABLE and ALTER TABLE.

☐ Write basic CREATE TABLE and ALTER TABLE statements and choose suitable types for identifiers, text, quantities, money, dates, and times.

☐ Use generated primary keys, foreign keys, NOT NULL, UNIQUE, DEFAULT, and CHECK to express the stated rules.

☐ Explain allowed or rejected changes, including missing values and invalid references; distinguish ON DELETE RESTRICT from ON DELETE CASCADE.

ER diagrams and relational mapping

☐ Identify entity sets, attributes, relationships, and keys; distinguish an entity attribute from a relationship attribute.

☐ Show one-to-one, one-to-many, and many-to-many relationships, and distinguish maximum cardinality from required participation.

☐ Read and draw the course's arrow notation; map basic ER relationships to relations with appropriate keys, uniqueness, and nullability.

Relational model and algebra reference

Term	Meaning
Relation	A set of tuples with the same attributes; no inherent tuple order or duplicate tuples.
Attribute and domain	An attribute is a named column; its domain is the set of allowed values.
Tuple	One record containing a value for each attribute.
Schema and instance	The schema describes structure and rules; an instance is the data stored at a particular time.
Superkey	An attribute set that identifies at most one tuple in every valid instance.
Candidate key	A minimal superkey: no proper subset is a superkey.
Primary and composite keys	The primary key is the chosen candidate key. A composite key uses more than one attribute.
Foreign key	Columns whose non-null references must match a referenced key. In our examples, that key is the parent table's primary key.

Keys follow the application's rules. A counterexample can disprove a proposed key; distinct values in today's rows do not prove uniqueness in every valid instance. Minimal does not necessarily mean the fewest attributes among all candidate keys.

Operators

R and S stand for relations; p and θ are conditions; A and B are attribute names. Read composed expressions from the inside out.

Operation	Notation	Effect
Selection	$\sigma_p(R)$	Keep tuples satisfying p; keep all input attributes.
Projection	$\pi_{A,B}(R)$	Keep the listed attributes and remove duplicate output tuples.
Rename	$\rho_x(R)$	Name the result x; do not change its tuples.
Cartesian product	$R \times S$	Form every pair of input tuples.
Natural join	$R \bowtie S$	Match on every shared attribute name; retain one copy of each shared attribute.
Theta join	$R \bowtie_\theta S$	Keep pairs satisfying θ; retain attributes from both inputs.
Union and intersection	$R \cup S$ $R \cap S$	Tuples in either input; tuples in both inputs.
Set difference	$R - S$	Tuples in R that are not in S.

Union, intersection, and difference require compatible schemas: the same number of attributes with compatible corresponding domains. Align names where needed. Natural join uses all shared names, not just declared keys; with no shared names it becomes a Cartesian product.

SQL syntax reference

Selection ordering and missing values

These short examples illustrate syntax. Here, checkout stores checkout_id, member_id, item_id, and returned_on; a missing return date means the item has not been returned.

```
SELECT checkout_id, item_id
FROM checkout
WHERE member_id = 1 AND returned_on IS NULL
ORDER BY checkout_id;
```

WHERE keeps only rows for which the condition is true. Use IS NULL or IS NOT NULL for missing values; ordinary comparisons with NULL produce unknown. Parentheses clarify mixed AND/OR conditions. ORDER BY defaults to ascending order; use DESC for descending.

Joins and duplicate removal

Assume member(member_id, name) and item(item_id, label). Join using columns that identify the intended partners; matching data types alone are not enough.

```
SELECT DISTINCT m.member_id, m.name
FROM member AS m
JOIN checkout AS c ON c.member_id = m.member_id
JOIN item AS i ON i.item_id = c.item_id
WHERE i.label = 'Camera';
```

AS introduces an alias. JOIN … ON keeps matching pairs. SELECT normally retains duplicate result rows; DISTINCT removes duplicates across the entire selected row. Adding a record ID to the output can make otherwise identical rows distinct. Row order is unspecified without ORDER BY.

Defining and changing tables

Data Definition Language (DDL) describes structure. This example assumes member(member_id integer PRIMARY KEY) already exists.

```
CREATE TABLE request (
    request_id integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    member_id integer NOT NULL
        REFERENCES member(member_id) ON DELETE RESTRICT,
    quantity integer NOT NULL DEFAULT 1
        CHECK (quantity BETWEEN 1 AND 4)
);
ALTER TABLE request ADD COLUMN note text;
```

A foreign key uses an existing parent ID; it does not generate that parent's ID. An added column is nullable unless declared otherwise. DROP COLUMN removes a column and its stored values. A composite primary key uses a table-level clause such as PRIMARY KEY (member_id, item_id), when the rules make that pair unique.

Common data types

integer: whole numbers. text: strings. boolean: true or false. date: calendar dates. timestamp: date and time; timestamptz: an instant with time-zone-aware input and display. numeric(p, s): exact decimal values with p total digits and s fractional digits, useful for money. Types and constraints work together to limit valid values.

Constraints and ER reference

SQL feature	What it does
PRIMARY KEY	Requires uniqueness and non-null values; may contain several columns.
FOREIGN KEY / REFERENCES	Requires a referenced key to exist. Add NOT NULL when the reference is required.
UNIQUE	Prevents repeated non-null key values or combinations; does not by itself require a value.
NOT NULL	Rejects missing values. Does not reject an empty or all-space text value.
CHECK (condition)	Rejects a row when the condition is false; unknown passes. Use for ranges, allowed values, or comparisons within a row.
DEFAULT value	Supplies a value when the column is omitted or DEFAULT is requested. Does not replace an explicit NULL.
ON DELETE RESTRICT	Blocks deletion of a referenced parent. The default NO ACTION also blocks it for the ordinary nondeferred constraints used here.
ON DELETE CASCADE	Deleting a parent automatically deletes child rows that reference it through this foreign key. Deleting a child does not delete its parent.

Useful checks include CHECK (price >= 0), CHECK (status IN ('open', 'closed')), CHECK (btrim(name) <> ''), and CHECK (ends_at > starts_at). Pair checks with NOT NULL when values are required. An ordinary row-level check does not compare a row with records in another table.

ER concepts and notation

An entity is one distinguishable object; an entity set groups objects of the same kind. An attribute describes an entity or relationship. A relationship associates entities. A relationship attribute describes the association, such as a student's grade in a particular section.

Symbol	Meaning
Rectangle / diamond	Entity set / relationship set.
Underlined attribute	Key attribute; underline all components of a composite key.
Arrow toward an entity set	At most one partner on that side. No arrow permits many.
Double / single line	Total participation (required) / partial participation (optional), for the entity set beside that line.
Dashed attribute connector	Attaches an attribute to a relationship.

Maximum cardinality and participation answer different questions. An arrow alone means at most one, not exactly one. Exactly one also requires total participation on the side whose entities must have that partner.

Mapping relationships to relations

Many-to-many: create a relation containing both participant keys and relationship attributes. When each pair occurs at most once, use both keys together as its primary key and each as a foreign key.

Many-to-one: place a foreign key on the many side. Add NOT NULL if every entity on that side must have a partner. Do not make that foreign key UNIQUE when several rows may share a partner.

One-to-one: use a UNIQUE foreign key to prevent sharing a partner; use NOT NULL when participation requires it. A foreign key alone does not require every referenced entity to have a matching referencing row.